אתם אורחים בסביבת העבודה Ajar's Workspace. קבלו גישה מלאה ל-Notion בסביבת העבודה שלכם.
הגדרת סביבת העבודה



פרויקט הגמר · 4255 · פלטפורמות מתקדמות לפיתוח ב-WEB ושימוש ב-AI

⚠️ **מועד ההגשה ומועד ההגנה: יפורסמו בקרוב** · ההגשה: קישור ל-repo **ציבורי** ב-GitHub, בתיבת ההגשה באתר המכללה. ההגנה: ב-Zoom.

🔄 **מה השתנה מאז שפרסמתי את ההוראות**

- **אין סרטון.** במקומו יש **הגנה אישית על הפרויקט**, בשיחה איתי של כ-7 דקות (סעיף 3).
- ניתוב עם **React Router**, שימוש ב **TypeScript**, ובדיקות עם **Vitest** הם עכשיו **חובה** ולא בונוס.
- **ההגשה היא קישור ל-repo ציבורי ב-GitHub**, בתיבת ההגשה באתר המכללה.
  לא zip ולא קבצים.
- **מי שכבר התחיל: פותחים repo חדש מהתבנית.** הכלים השתנו, אז כולם מתחילים מחדש. את הרעיון ואת ה-`PRD.md` לוקחים איתכם. את הקוד בונים מחדש, משימה אחרי משימה — זה התהליך שנבדק.

🚀 **מתחילים מכאן:** מחוברים ל-GitHub, פתחו יצירת repo חדש מהתבנית ← בוחרים **Create repository** ← **Public**. ההמשך ב-README של התבנית.

יש בו דבר אחד: רישום **אוטומטי** של כל פרומפט שאתם שולחים לסוכן, אל תוך `PROMPTS.md`. את אפליקציית Vite יוצרים בתוכו, לפי ה-README שלו.

למה יש הגנה על הפרויקט

אתם נכנסים לתקופה שבה את רוב הקוד לא תכתבו בעצמכם. סוכן קידוד יכתוב אותו, וזה בסדר — בקורס הזה זה גם מה שמצופה מכם. אבל ה-codebase עדיין **שלכם**, ואתם אחראים עליו: איך הוא בנוי, איך החלקים מדברים זה עם זה, ומה יקרה אם משנים שורה. מפתח שלא יודע לענות על השאלות האלה על הקוד של עצמו, לא באמת שולט בו.

לכן הפרויקט נבדק בשני שלבים. **ראשית**, הוא צריך לעבוד ולעמוד בכל הדרישות שלמטה. **שנית**, אתם צריכים להכיר אותו מבפנים ולהגן עליו בשיחה איתי. יש לכם סוכן זמין 24/7 — השתמשו בו גם כדי **ללמוד** את הפרויקט שהגשתם: שאלו אותו על כל קובץ, כל רכיב וכל החלטה, עד שאתם יכולים להסביר אותם בעצמכם.

1. מה בונים

אפליקציית master-detail עם כמה עמודים: רשימת פריטים, עמוד פרטים לכל פריט, ולפחות **שני עמודים נוספים** שאתם מתכננים בעצמכם.

- פרוייקט front-end, **TypeScript + Vite + React** בלבד. אין שרת, אין בסיס נתונים.
- ניתוב עם **React Router**: כתובת נפרדת לכל עמוד, וכתובת נפרדת לכל פריט (למשל `items/25/`).
- בדיקות יחידה (unit tests) עם **Vitest**.
- המידע מגיע מ-**API ציבורי חינמי בלי מפתח**.
- **הנושא לבחירתכם.** פוקימונים, מתכונים, ספרים, אנימה - מה שמעניין אתכם.

רעיונות לשני העמודים הנוספים: מועדפים · השוואה בין שני פריטים · סטטיסטיקות על הרשימה · פריט אקראי · היסטוריית צפייה. עמוד "אודות" עם טקסט בלבד לא נחשב: בכל עמוד המשתמש צריך לעשות משהו או לראות מידע.

🎯 **למה דווקא master-detail?** כי התבנית הזאת מכסה בדיוק את מה שלמדנו: פירוק לרכיבים, props, state, רשימות עם `map`, ומשיכת מידע מהרשת. TypeScript, ניתוב ובדיקות מוסיפים את מה שיש כמעט בכל פרויקט React אמיתי.

2. דרישות חובה

- [ ] פרויקט **TypeScript + Vite + React** שרץ עם `npm run dev` בלי שגיאות בקונסול, ו-`npm run build` עובר בלי שגיאות.
  יוצרים אותו בתוך ה-repo מהתבנית:
  `npm create vite@latest . -- --template react-ts`.
- [ ] **טיפוסים** (`type` או `interface`) למידע שמגיע מה-API ול-props של כל רכיב. בלי `any`.
- [ ] לפחות **4 components** משלכם, מלבד העמודים, כל אחד בקובץ נפרד.
- [ ] **ניתוב עם React Router**: עמוד רשימה, עמוד פרטים עם כתובת משלו (`/items/:id`), ולפחות **שני עמודים נוספים**. תפריט ניווט בין העמודים עם `Link` או `NavLink`.
- [ ] רשימה שנבנית עם `map`, עם `key` ייחודי לכל פריט.
- [ ] **לחיצה על פריט פותחת את הפרטים שלו**, וה-`id` נקרא מהכתובת עם `useParams`. מידע עובר בין רכיבים ב-props.
- [ ] המידע מגיע מ-**API ציבורי** עם `fetch` בתוך `useEffect`.
- [ ] **שלושה מצבים על המסך**: טוען · שגיאה · יש מידע.
- [ ] **חיפוש או סינון** של הרשימה.
- [ ] **בדיקות יחידה** (unit tests) **עם Vitest**: לפחות **5 בדיקות** שעוברות עם `npm test`, על לוגיקה שלכם — למשל הסינון, או עיבוד המידע שמגיע מה-API. את הבדיקות מריצים ב-**Vitest UI**, בדפדפן: בקשו מהסוכן להתקין את `vitest/ui@` ולהוסיף סקריפט `test:ui`, ואז `npm run test:ui`.
- [ ] משלכם. האפליקציה נראית סבירה גם במסך צר CSS.
- [ ] `README.md`: מה האפליקציה עושה, איך מריצים אותה ואת הבדיקות, ובאיזה API השתמשתם.
- [ ] אחרי כל משימה — commit **הדרגתיים** עם GitHub-**ציבורי** ב repo.
- [ ] `PRD.md`, `tasks.md`, `PROMPTS.md` בתוך ה-repo. `PROMPTS.md` נכתב לבד — רק תזכרו לעשות לו commit.

`PRD.md` ו- `tasks.md` — מה הם ולמה הם שם

- `PRD.md` (Product Requirements Document) מתאר **מה** בונים ו**למה**: למי האפליקציה, אילו עמודים יש בה, ומה המשתמש רואה בכל אחד ("When I..., I see..."). זה המסמך שאתם והסוכן חוזרים אליו כדי לדעת מתי משהו "גמור".
- `tasks.md` הוא תוכנית העבודה: משימות בסדר בנייה, לכל אחת שורת "Done when:". סוכן עובד הכי טוב על משימה אחת קטנה בכל פעם.
- **מסמכים נוספים — מותר ורצוי.** למשל `docs/architecture.md`: מבנה הרכיבים והעמודים, ואיך המידע זורם ביניהם. קשרו אליהם מתוך `PRD.md`.

מה לא נדרש (ועדיף בלי)

לא צריך	למה
שרת, בסיס נתונים, הרשמה והתחברות	לא בהיקף הקורס. זה גם מה שמפיל פרויקטים באמצע.
Redux, Next.js, ספריות UI כבדות	`useState` מספיק לאפליקציה בגודל הזה.
API בתשלום או כזה שדורש מפתח	מפתח בקוד צד-לקוח = מפתח חשוף לכולם.

בונוס, למי שרוצה יותר (לא מעלה את הציון מעל 100, אבל נחשב באיכות): שמירת מועדפים ב-`localStorage` · בדיקות לרכיבים עם React Testing Library · עמוד 404 לכתובת שלא קיימת.

3. ההגנה על הפרויקט

- **מתי:** המועדים יפורסמו בקרוב.
- **איפה:** ב-Zoom בלבד.
- **כמה זמן:** כ-7 דקות לכל סטודנט.
- **שיבוץ:** גיליון שיבוץ יפורסם ב-Classroom. כל אחד רושם את עצמו למשבצת זמן.
- **מה להכין:** הפרויקט פתוח ב-editor ורץ מקומית, מצלמה פתוחה, ומוכנים לשתף מסך.

- **מה יקרה:** אשאל כמה שאלות על הפרויקט **שלכם**. בכיוון של השאלות ששאלתי בשיעורים, אבל לא רק על הנושאים שלמדנו. למשל: איפה נשמר ה-state ולמה דווקא שם · מה קורה מהלחיצה על פריט ועד שהפרטים מופיעים · איך מוגדרים העמודים והכתובות · מה בודקת בדיקה מסוימת · מה הייתם משנים כדי להוסיף יכולת חדשה. ייתכן שאבקש לפתוח קובץ ולהראות איפה דבר מסוים קורה.
- **בזמן ההגנה עונים בעצמכם**, בלי סוכן ובלי חיפוש.

> 🧠 **איך מתכוננים:** השתמשו בסוכן כדי ללמוד את הפרויקט שהגשתם. בקשו ממנו להסביר כל קובץ, לשרטט את מבנה הרכיבים והעמודים, ואז לשאול **אתכם** שאלות על הקוד — עד שאתם עונים בלי עזרה.

4. הניקוד

אין מבחן. הציון כולו מההגנה עליו:

חלק	נק'	מה בודקים
הפרויקט	**60**	אפליקציה שעובדת, קוד מסודר, שימוש בנושאי הקורס
· האפליקציה עובדת	18	רשימה, פרטים, שני העמודים הנוספים, חיפוש/סינון. בלי קריסות ובלי שגיאות בקונסול
· React ו-TypeScript	12	פירוק הגיוני לרכיבים, state במקום הנכון, props נקיים, `key` תקין, טיפוסים בלי `any`
· ניתוב	8	כתובת לכל עמוד ולכל פריט, `useParams`, תפריט ניווט
· עבודה עם API	8	`fetch` ב-`useEffect`, מצבי טעינה ושגיאה
· בדיקות	8	לפחות 5 בדיקות Vitest שעוברות, על לוגיקה אמיתית ולא על דברים טריוויאליים
· קוד ועיצוב	6	שמות ברורים, קבצים מסודרים, CSS משלכם, נראה טוב גם במסך צר
תהליך העבודה	**20**	איך הגעתם לתוצאה, לא רק התוצאה
· commits הדרגתיים	8	היסטוריה שמספרת סיפור. commit אחד ענק = אין נקודות
· `PRD.md` ו-`tasks.md`	6	שורות קבלה שאפשר לבדוק בעיניים, משימות בסדר בנייה, קישור למסמכים נוספים אם יש
· `PROMPTS.md`	6	נרשם אוטומטית על ידי ה-hook שבתבנית. נבדק התהליך: ניסיון, בדיקה, תיקון — ולא פרומפט אחד ענק
ההגנה על הפרויקט	**20**	שתוכלו להסביר כל חלק בקוד שלכם
· תשובות בשיחה (כ-7 דקות)	20	מבנה הפרויקט, זרימת המידע, ניתוב, בדיקות, וההחלטות שקיבלתם

⚖️ **ההגנה מאשרת גם את הניקוד על הפרויקט.** הנקודות על חלק בפרויקט ניתנות רק אם אתם מסבירים אותו בהגנה. פרויקט שעובד מצוין, אבל אתם לא יודעים להסביר איך, לא יקבל את מלוא 60 הנקודות.

🤖 **עבודה עם סוכן AI מותרת ומומלצת.** מה שנבדק זה לא מי כתב את השורה, אלא אם אתם מבינים אותה: יכולים להסביר אותה, למצוא אותה בקובץ, ולשנות אותה בלי הסוכן.

⛔ **מה מוריד ציון**

- repo עם commit אחד בסוף, או עם קוד שהועתק בבת אחת.
- קוד שאי אפשר להסביר בהגנה.
- `npm run build` או `npm test` נכשלים. שימוש ב- `any` .
- העתקה של אפליקציית הדוגמה `countries-explorer` עם החלפת שמות.
- API שדורש מפתח, או מידע שהודבק ידנית במקום להגיע מה-API.
- repo קובץ zip, פרטי, או קישור שלא נפתח.

5. רעיונות לפרויקט

כל הרעיונות למטה הם master-detail על API שנבדק ועובד. אפשר גם רעיון משלכם — רק שיישאר באותו היקף.

#	הרעיון	API	ברשימה	בפרטים
1	פוקידקס	PokéAPI	שם + תמונה	סוגים, יכול
2	מדריך סדרות	TVmaze	שם + פוסטר	ז'אנרים, די
3	דמויות ריק ומורטי	Rick & Morty	שם + תמונה	סטטוס, מין
4	ספר מתכונים	TheMealDB	שם המנה + תמונה	מצרכים, ה
5	חיפוש ספרים	Open Library	כותרת + מחבר	שנה, נושא
6	גלריית מוזיאון	Art Institute of Chicago	יצירה + תמונה	אמן, שנה,
7	מדריך גזעי כלבים	Dog CEO	שם הגזע	גלריית תמ
8	מדף אנימה	Jikan (MyAnimeList)	כותרת + פוסטר	מספר פרק
9	מלחמת הכוכבים	SWAPI	שם הדמות	גובה, מין, כ
10	עולם הארי פוטר	HP-API	שם + תמונה	בית, שרביט
11	אספן קלפי Magic	Scryfall	שם הקלף + תמונה	עלות, סוג,
12	דמויות דיסני	Disney API	שם + תמונה	סרטים, סד
13	מזג אוויר בערים שלי	Open-Meteo	רשימת ערים שאתם מגדירים	טמפרטורה

> 💡 **עוד השראה:** app-ideas — רשימה גדולה של רעיונות לאפליקציות, עם דרישות לכל אחת. קחו משם רק רעיונות מדרגת **Beginner** או **Tier-2** שהם front-end בלבד, למשל *Cause Effect App*, *Book Finder*, *Recipe App*, *GitHub Profiles*, *Podcast Directory*. התאימו אותם לתבנית רשימה + פרטים.

6. APIs חינמיים בלי מפתח

כל הכתובות למטה נבדקו ב-**23/09/2026**: החזירו מידע תקין, ומאפשרות קריאה מהדפדפן (CORS פתוח).

נושא	כתובת
פוקימונים	`https://pokeapi.co/api/v2/pokemon?limit=60`
סדרות טלוויזיה	`https://api.tvmaze.com/shows`
ריק ומורטי	`https://rickandmortyapi.com/api/character`
מתכונים	`https://www.themealdb.com/api/json/v1/1/search.php?s=pasta`
ספרים	`https://openlibrary.org/search.json?q=harry+potter`
יצירות אמנות	`https://api.artic.edu/api/v1/artworks`
גזעי כלבים	`https://dog.ceo/api/breeds/list/all`
אנימה	`https://api.jikan.moe/v4/top/anime`
מלחמת הכוכבים	`https://swapi.info/api/people`
הארי פוטר	`https://hp-api.onrender.com/api/characters`
קלפי Magic	`https://api.scryfall.com/cards/search?q=dragon`
דמויות דיסני	`https://api.disneyapi.dev/character`
מזג אוויר	`https://api.open-meteo.com/v1/forecast?latitude=32.08&longitude=34.78&current_weather=true`
משתמשים לדוגמה	`https://jsonplaceholder.typicode.com/users`

אם בקונסול מופיעה שגיאת CORS — ה-API הזה לא מרשה קריאה ישירה מהדפדפן. אל תחפשו proxy ואל תתקינו תוספים: פשוט החליפו ל-API אחר מהרשימה למעלה.